

Handling Missing Data: Categorical Variables

Module 37 Study Notes • Feature Engineering

1. Introduction to Categorical Imputation

In the previous module, we explored filling missing *numerical* data using the Mean or Median. However, you cannot calculate the average of text strings like "Mumbai" and "Delhi". When dealing with missing categorical data, we must switch our strategy. There are two primary techniques for univariate categorical imputation.

2. Strategy 1: Most Frequent Imputation (Mode)

This is the categorical equivalent of Mean/Median imputation. You calculate the **Mode** (the category that appears most frequently in the column) and replace every single missing value with that category.

Example: If 80% of your users are from "Mumbai," you assume that any user who left the "City" field blank is also from Mumbai.

The Constraints (When is this safe?)

- **Missing Completely At Random (MCAR):** Just like numerical imputation, this strictly requires the data to be missing randomly.
- **The 5% Rule:** You should only apply this if less than 5% of the data is missing.
- **The Dominance Rule:** This technique is only mathematically safe if one category heavily dominates the others. If "Mumbai" is 34%, "Delhi" is 33%, and "Kolkata" is 33%, replacing 5% of missing values with "Mumbai" will artificially unbalance the dataset.

Implementation in Scikit-Learn

```
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer

# 1. Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# 2. Define the Imputer using the 'most_frequent' strategy
impute_mode = SimpleImputer(strategy='most_frequent')
```

```
# 3. Add to ColumnTransformer
ct = ColumnTransformer(
    transformers=[
        ('impute_city', impute_mode, ['City'])
    ],
    remainder='passthrough'
)

# 4. Fit and Transform
X_train_transformed = ct.fit_transform(X_train)
```

3. Strategy 2: Missing Category Imputation (Constant)

What happens if your data violates the 5% rule? What if 30% of your column is missing? If you replace 30% of your data with "Mumbai", you destroy the underlying patterns of the dataset.

In cases of high missingness, or when data is *not* missing randomly, we use **Missing Category Imputation**. Instead of guessing what the value was supposed to be, you explicitly tell the machine learning algorithm that the value was missing by creating a brand new category called **"Missing"**.

Why does this work?

Sometimes, the fact that a user refused to answer a question is a powerful predictive signal in itself. By creating a specific "Missing" category, the algorithm can learn the behavioral pattern of people who leave forms blank.

Implementation in Scikit-Learn

```
from sklearn.impute import SimpleImputer

# 1. Define the Imputer using 'constant' strategy and a specific fill_value
impute_missing = SimpleImputer(strategy='constant', fill_value='Missing')

# 2. Add to ColumnTransformer (same as above)
ct = ColumnTransformer(
    transformers=[
        ('impute_high_missing_col', impute_missing, ['Income_Bracket'])
    ],
    remainder='passthrough'
)

# 3. Fit and Transform
```

```
X_train_transformed = ct.fit_transform(X_train)
```

4. Summary of Strategy Selection

- If missing data is < **5%** AND one category is highly dominant $\rightarrow$ Use **Most Frequent (Mode)**.
- If missing data is > **5%** OR there is a known pattern to the missingness $\rightarrow$ Use **Missing Category ('Constant')**.